Voice Education Assistant — Technical Documentation

NITDA/NCAIR National AI Innovation Challenge 2026 Problem Statement 2 — Voice-First Access Domain: Education

Repository: https://github.com/khalifa-4619/voice-education-assistant **Video demonstration:** see `docs/SUBMISSION_URLS.txt`

Executive summary

The Voice Education Assistant is a voice-first AI tutor for Nigerian secondary-school students. A student presses a microphone button, asks a question aloud, and receives a spoken answer. No typing. No reading long text. No accounts. No internet required.

The system runs entirely on a single CPU-only laptop. It uses the official NCAIR N-ATLAS models for both speech recognition and answer generation. It has been validated with 52 real interactions recorded on the target hardware, with a self-reported useful-answer rate of 85%.

This document covers:

1. The problem this addresses
2. The architecture
3. The N-ATLAS integration
4. Setup and usage
5. Real measured performance
6. Language coverage and limitations
7. Validation methodology and results
8. Path to production

---

1. The problem

Nigerian secondary-school students often lack affordable, on-demand access to a subject tutor. Text-based AI tools are helpful for students who are comfortable typing and reading in English, but they exclude students who are more comfortable speaking, students with limited literacy in the standard written register, and students whose primary language is not English.

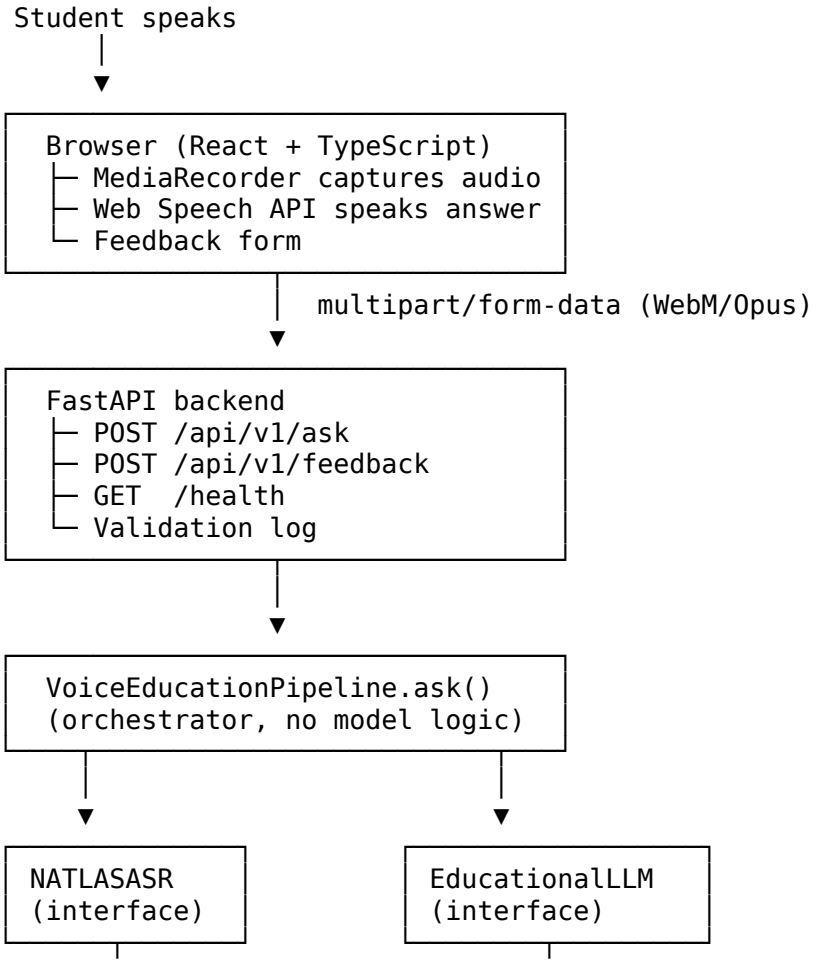
The problem statement for this challenge — Voice-First Access — asks for a solution that reaches these students through speech. The Voice

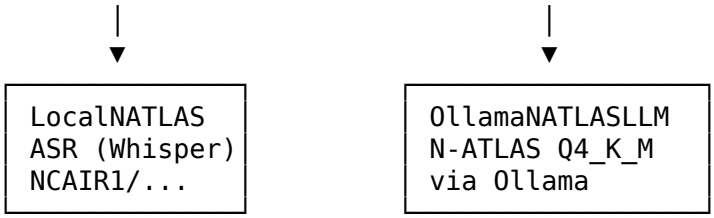
Education Assistant is designed against that constraint:

- **Voice input** — no keyboard required.
- **Voice output** — no screen required to receive the answer.
- **Runs on modest hardware** — a laptop with 14 GB RAM, no GPU.
- **Works offline** — after the models are cached, no network is needed.
- **Responds in the language used** — English verified end-to-end; Hausa verified at the LLM layer, with the ASR side designed to swap in the matching NCAIR1 model.

---

2. Architecture





The pipeline depends on interfaces, not implementations. The frontend does not know which model is behind `/api/v1/ask`. Swapping to the official NCAIR hosted API is a change to `backend/app/dependencies.py` alone.

2.1 Layers and their responsibilities

Layer	Responsibility	Location
Frontend	Capture audio, render answer, speak answer, collect feedback	`frontend/src/`
HTTP API	Validate uploads, transform between HTTP and pipeline, log interactions	`backend/app/api/`
Pipeline	Orchestrate ASR then LLM, record timings	`backend/app/pipeline.py`
Adapters	Encapsulate each model behind a stable interface	`backend/app/asr/`, `backend/app/llm/`
Composition	Decide which concrete adapters the running app uses	`backend/app/dependencies.py`
Config	Environment-driven settings	`backend/app/config.py`
Validation	Append-only interaction log	`backend/app/validation/`

2.2 Why this shape

Three engineering decisions shape the code:

1. **Adapters behind interfaces.** Both N-ATLAS ASR and N-ATLAS LLM are defined by single-method abstract base classes. The

pipeline imports only the interfaces. This makes the app testable without models, and swappable without rewriting.

2. **Composition root, not scattered selection.** A single file (`dependencies.py`) reads environment variables and constructs the pipeline. The route handlers never inspect provider names. This is what allows a local CPU path, a Colab path, and a future hosted path to share the same code.
3. **Dataclasses for internals, Pydantic for HTTP.** The pipeline returns a frozen dataclass. The HTTP layer converts that to a Pydantic model at the boundary. Neither is burdened with the other's concerns.

---

3. N-ATLAS integration

Full evidence: see `docs/NATLAS_INTEGRATION.md`.

3.1 Models used

Component	Model ID
ASR	`NCAIR1/NigerianAccentedEnglish` (Whisper Small)
LLM	`NCAIR1/N-ATLaS` (Llama-3 8B fine-tune)
LLM quantized	`hf.co/Tushe/AMINI-ASSISTANT-GGUF-Q4-B:Q4_K_M`

The Q4_K_M variant is a community conversion with the original weights unchanged, necessary to run an 8B model on a CPU-only laptop. The full FP16 model was also verified on a Colab T4 in earlier work.

3.2 Invocation path

- **ASR:** `transformers.WhisperProcessor` + `WhisperForConditionalGeneration` running locally.
- **LLM:** HTTP POST to a local Ollama server at `http://127.0.0.1:11434/api/generate`.

Both paths are documented in `docs/NATLAS_INTEGRATION.md`, with the specific code excerpts.

---

4. Setup and usage

4.1 Prerequisites

- Python 3.11+ (verified on 3.12 and 3.13)
- Node.js 18+ (verified on 24)
- FFmpeg (for audio format conversion)
- Ollama (for local N-ATLAS LLM inference)
- ~7 GB free disk for models

4.2 Backend

```
cd backend
python -m venv .venv
.\.venv\Scripts\Activate.ps1        # Windows
# source .venv/bin/activate          # Linux / macOS
pip install -r requirements.txt
```

4.3 Models

```
# N-ATLAS LLM (via Ollama)
ollama run hf.co/Tushe/AMINI-ASSISTANT-GGUF-Q4-B:Q4_K_M

# N-ATLAS ASR (via Hugging Face; requires accepting the model licence)
# Downloaded automatically on first use by transformers.
```

4.4 Run the backend

```
cd backend
$env:ASR_PROVIDER = "local"
$env:LLM_PROVIDER = "ollama"
uvicorn app.main:app --port 8000
```

4.5 Run the frontend

```
cd frontend
npm install
npm run dev
```

Then open `http://localhost:5173/` in a modern browser.

4.6 Configuration

Variable	Default	Purpose
`ASR_PROVIDER`	`fake`	`fake` | `local` | `hosted`
`LLM_PROVIDER`	`fake`	`fake` | `local` | `ollama` | `hosted`
`OLLAMA_BASE_URL`	`http://127.0.0.1:11434`	Ollama server
`OLLAMA_MODEL`	`hf.co/Tushe/AMINI-ASSISTANT-GGUF-Q4-B:Q4_K_M`	AMINI-ATLAS GGUF
`MAX_AUDIO_SIZE_MB`	`10`	Upload limit
`ALLOWED_AUDIO_TYPES`	`wav,mp3,m4a,webm,ogg,flac`	Accepted formats

4.7 Tests

```
cd backend
python -m pytest -v
```

32 tests covering the pipeline orchestrator, the HTTP endpoints, the validation logging, the composition root, and the audio-format helper.

4.8 API reference

```
GET  /health                → { status, providers }
POST /api/v1/ask       → multipart audio → { interaction_id, transcript,
                                  answer, timings, started_at }
POST /api/v1/feedback    → JSON { interaction_id, useful, language,
                             subject, notes } → { recorded }
```

Interactive API docs are available at `http://127.0.0.1:8000/docs`.

---

5. Real measured performance

All numbers below were measured on the target hardware during the validation session and are recorded in `docs/validation/interactions.jsonl`.

Metric	Value
Average ASR time	**14.4 s**
Average LLM time	**48.0 s**
Average total per interaction	**62.5 s**
Fastest interaction	**13.7 s**
Slowest interaction	**278.1 s**
Hardware	4-core / 8-thread CPU, 14 GB RAM, no GPU

The spread is expected: short factual answers complete quickly, while long explanatory answers can take several minutes. Latency is a hardware constraint, not an architectural one. The pipeline itself is fast — the time is dominated by autoregressive token generation on CPU.

---

6. Language coverage and limitations

Language	Speech input (ASR)	Text answer (LLM)	Spoken answer (TTS)
Nigerian-accented English	Verified end-to-end	Verified	Verified
Hausa	Requires model swap	Verified separately	No Hausa voice on Windows

Nigerian-accented English is fully validated: the ASR model was fine-tuned by NCAIR for this accent.

Hausa has been verified at the LLM layer: N-ATLAS produces coherent Hausa educational answers when prompted in Hausa (see `docs/results/natlas_llm_poc_hausa_result.json`). Speech input in Hausa requires swapping the ASR model to `NCAIR1/Hausa-ASR`, which is a one-line change in `backend/app/dependencies.py`. This was not done during the validation session because the session prioritized a complete English end-to-end run within the available time.

Text-to-speech uses the browser's built-in Web Speech API. Windows does not ship a Hausa voice by default, so a Hausa answer would be pronounced with English phonetics. Replacing the TTS with a proper multilingual service is a single-file change (`frontend/src/tts.ts`).

These limitations are documented because we prefer an honest statement over an unverified claim.

---

7. Validation methodology and results

Full report: `docs/validation/SUMMARY.md`

7.1 Method

- 52 real interactions recorded in one session on the target laptop.
- Each interaction: one question spoken into the browser, one answer returned by the N-ATLAS pipeline, one optional usefulness rating.
- Every interaction is logged as a single JSON line in `docs/validation/interactions.jsonl`.

7.2 Results

Metric	Value
Interactions recorded	**52**
With student feedback	47
Rated useful	40
Rated not useful	7
Useful rate	**85%**

7.3 Subjects covered

- General: 26
- Social studies: 8
- Mathematics: 6
- English: 3
- Science: 1
- Other: 3

7.4 Representative examples

Successful interaction: > *Question (transcribed):* "do you describe the concept of a life cycle?" > *Answer:* A structured description of plant and animal life cycles. > *Timings:* ASR 20.1s · LLM 38.2s · Total 58.3s · Rated useful.

Weaker interaction (honest failure): > *Question (transcribed):* "what's the spectacularest theory?" > *Answer:* "I'm sorry, I don't understand the question. Could you provide > more details or rephrase it?" > *Timings:* ASR 22.5s · LLM 29.2s · Total 51.7s · Rated not useful.

The second example is valuable: it shows the model correctly declining to guess when the transcribed input was ambiguous, rather than hallucinating an answer. This is the desired behaviour for an educational assistant.

---

8. Path to production

The current implementation is a validated prototype, not a production deployment. It runs on one laptop. Scaling to a school or a region requires three changes, all of which the current architecture already supports:

1. **Hosted N-ATLAS inference.** The challenge provides hosted N-ATLAS API credentials to shortlisted teams. `HostedNATLASLLM` is already scaffolded in `backend/app/llm/hosted.py`; enabling it is a change to `dependencies.py` only. This alone would reduce per-interaction latency from ~60 s to a few seconds.
2. **Multilingual ASR and TTS.** Swapping the ASR model ID to `NCAIR1/Hausa-ASR` (or Yoruba, or Igbo) activates Hausa speech input. A production TTS service (e.g. a hosted Hausa voice) replaces the browser TTS in one file.
3. **Deployment.** The FastAPI backend is containerizable; the React frontend is a static build. Neither has any hard dependency on the development machine.

None of these require changing the pipeline, the frontend, or the validation infrastructure. They are swaps at the adapter boundary, which is exactly what the architecture was designed for.

---

9. Evidence summary

Evidence	Location
Source code	https://github.com/khalifa-4619/voice-education-assistant
32 automated tests	`backend/tests/`
52 validation interactions	`docs/validation/interactions.jsonl`
Validation summary	`docs/validation/SUMMARY.md`
N-ATLAS integration evidence	`docs/NATLAS_INTEGRATION.md`
Verification milestones	`docs/results/*.json`
Video demonstration	see `docs/SUBMISSION_URLS.txt`

“‘